



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**Design a smart-city emergency routing system with 50+ intersections
(Nodes) and 100+ weighted roads (Edges and Weights)**

Submitted in the partial fulfilment for the Course

of

CSA0311- Data Structures

to the award of the degree of

BACHELOR OF ENGINEERING

IN

BTECH IN COMPUTERSCIENCE AND BIOSCIENCES

Submitted by

K.YOGANAND(192525173)

K.NIKHIL(192525307)

P.JAI AKASH(192521353)

Under the Supervision of

Dr.K.Kiruthika

Saveetha School of engineering

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

September 2026

COMMON COURSE ASSIGNMENT FORMAT

Assignment Information

Particular	Details
Department	Computer Science and Engineering (Computer Science and Biosciences)
Programme	B.E. / B.Tech – Computer Science and Bioscience Engineering
Course Code & Course Name	CSA0311 – Data Structures
Academic Year / Batch	2025 – 2026
Faculty Name	Dr. K. Kiruthika
Assignment Title	Design a smart-city emergency routing system with 50+ intersections (Nodes) and 100+ weighted roads (Edges and Weights)
Date of Issue	31/August/2026
Date of Submission	2/September /2026
Maximum Marks	100
Course Outcome(s) – CO	Apply appropriate data structures and graph algorithms to solve a real-world computing problem
Bloom's Taxonomy Level	L3 / L4 / L5 (Apply, Analyze, Evaluate)
SDG Mapping (SDG 1-17)	SDG 9 – Industry, Innovation & Infrastructure; SDG 11 – Sustainable Cities and Communities
Industry / Societal Relevance	Smart-city traffic management and emergency-response infrastructure

Problem Statement

In smart-city transportation networks, emergency response vehicles (ambulances, fire engines, police units) must navigate complex urban road grids under rapidly fluctuating traffic conditions. Delays caused by sub-optimal routing or sluggish path re-computation can cost human lives. A smart-city traffic management authority requires a real-time emergency routing engine that ingests road-network topology (intersections, roads, distances, base travel times) together with continuous live sensor feeds (congestion, construction, accidents) and returns the shortest available route to an incident within a strict latency budget, while remaining memory-efficient enough for embedded deployment.

Requirements and Constraints

Six non-negotiable operational constraints were identified for the system, covering functional, performance, memory and flexibility requirements relevant to this course problem:

No.	Constraint	Required Specification	Technical Solution Adopted
1	Network scale	>5,000 intersections, >10,000 roads	Hash-indexed sparse adjacency list
2	Real-time latency	Query time < 200 ms	Binary min-heap Dijkstra with lazy deletion
3	Optimal route	Weighted shortest travel time	Dijkstra greedy invariant with prev[] back-tracing
4	Live traffic updates	Frequent dynamic weight changes	O(1) composite-key edge hash table
5	Moderate memory	Low RAM / embedded-friendly	Sparse O(V+E) storage instead of O(V ²) matrix
6	Query flexibility	1-to-1 and repeated 1-to-many queries	Decoupled DijkstraResult architecture

Student Work

Problem Understanding and Formulation

The problem is to design and implement a real-time emergency-vehicle routing engine for a smart city road network containing more than 5,000 intersections and more than 10,000 directed roads. The expected outcome is a system that, given a source and a destination, returns the minimum-cost route together with its total travel time within a 200 ms latency budget, and that can absorb live traffic updates without recomputing the entire graph.

The information available consists of the road-network topology (named intersections and directed, weighted roads representing travel time) and a live feed of traffic events that change individual road weights. The key assumptions made are that all road weights are strictly positive (no negative edges), that the graph may be disconnected in places, and that queries may be issued repeatedly between the same or different source-destination pairs. The constraints to be satisfied are the six listed above: scale, latency, optimality, live updates, memory footprint and query flexibility.

Application of Course Knowledge

The solution applies core Data Structures & Algorithms concepts: graph representation, priority queues, hashing, and asymptotic complexity analysis, to a single high-performance C system.

Graph representation – adjacency matrix vs. hash-indexed adjacency list

For $V = 5,000$ intersections, a dense adjacency matrix requires $5,000 \times 5,000 \times 8 \text{ bytes} \approx 190.7 \text{ MB}$ of memory, of which about 99.88% of cells are empty because a typical intersection connects to only 3–6 roads. Scanning neighbours for Dijkstra's relaxation step then costs $O(V)$ per popped node, i.e. $O(V^2)$ overall. A hash-indexed sparse adjacency list instead stores only the edges that actually exist: $5,000 \times 8 \text{ B}$ (node array) + $5,000 \times 8 \text{ B}$ (name array) + $30,000 \times 32 \text{ B}$ (edge structs) + node/edge hash tables (15,013 and 20,011 prime-sized buckets) $\approx 1.15 \text{ MB}$ in total — roughly 165× less memory — while neighbour lookup drops to $O(\deg(u)) \approx 3\text{--}6$ pointer hops.

Shortest-path algorithm selection

Algorithm	Time Complexity	Weighted?	Verdict
Breadth-First Search	$O(V+E)$	No	Rejected – only counts hops, ignores travel-time weights
Floyd-Warshall	$O(V^3)$	Yes	Rejected – $5000^3 \approx 1.25 \times 10^{11}$ ops, infeasible in real time
Bellman-Ford	$O(V \cdot E)$	Yes	Rejected – built for negative weights, unnecessary here (150–300 ms)
Dijkstra (linear-scan PQ)	$O(V^2+E)$	Yes	Rejected – $O(V)$ min-search per step breaches the 200 ms budget
Dijkstra + Binary Min-Heap	$O((V+E) \log E)$	Yes	Selected – solves 5,000 nodes in 2–3 ms

Priority queue design – lazy-deletion heap vs. indexed decrease-key

When a shorter distance to a node is discovered, an indexed decrease-key heap must locate and mutate the existing heap entry in place using an auxiliary position array, adding indirection and cache misses. The implemented lazy-deletion binary min-heap instead simply pushes a new (node, distance) pair; a stale entry is discarded in $O(1)$ when popped if the node is already finalised. The asymptotic cost rises marginally from $O((V+E) \log V)$ to $O((V+E) \log E)$ — for $E = 30,000$ this is $\log_2(30,000) \approx 14.8$ versus $\log_2(5,000) \approx 12.2$, a difference of well under 0.1 ms — in exchange for a much simpler, cache-friendlier heap implementation.

Hashing for O(1) live traffic updates and node lookup

Road weights change due to congestion, accidents and closures. A direct (source, destination) $\rightarrow$ Edge* composite-key hash table, sized to a prime number of buckets (20,011) with separate chaining, allows any road's weight to be located and overwritten in O(1) average time. A similar djb2-style hash map converts intersection names to contiguous integer indices, and visited-node status during Dijkstra's inner loop is tracked with a direct-mapped byte array (O(1) read/write, zero collisions, zero extra allocation).

Solution / Design / Methodology

The system, PulseRoute, is organised as four layers: (1) a CSV-based road-network ingestion layer; (2) a hash-indexed adjacency-list graph-storage core with O(1) live weight mutation; (3) a Dijkstra + lazy-deletion min-heap pathfinding engine that produces a reusable DijkstraResult (a single solve can be replayed for many destinations in O(hops) each); and (4) a query/dispatch layer exposing both a CLI and a REST API / web command-centre dashboard, protected by a cross-platform reader-writer lock so that concurrent clients can read the graph while a single writer applies traffic updates.

File	Responsibility	Key Functions / Structures
graph.h / graph.c	Hash-indexed adjacency list & O(1) live traffic engine	graph_create, graph_add_edge, graph_update_traffic, graph_get_node_index
minheap.h / minheap.c	Binary min-heap priority queue with lazy deletion	heap_create, heap_push, heap_pop, sift_up, sift_down
dijkstra.h / dijkstra.c	Single-source shortest-path solver & path reconstruction	dijkstra_run, find_shortest_path, path_from_result
routing_lock.h	Cross-platform reader-writer lock	routing_rwlock_init/rdlock/wrlock

csv_loader.h / csv_loader.c	Road-network CSV ingestion	load_graph_csv
service.h / service.c	Embedded HTTP server, siren synthesiser & dashboard UI	start_http_service, handle_client_request
main.c	CLI, scale benchmark and 100-route test-suite runner	main, demo_scale_benchmark, demo_100_routes

Pseudocode

DIJKSTRA(Graph, source, destination)

 for each vertex v

$\text{dist}[v] = \text{INFINITY}$

$\text{parent}[v] = -1$

$\text{visited}[v] = \text{false}$

$\text{dist}[\text{source}] = 0$

 insert (source, 0) into MinHeap

 while MinHeap is not empty

$u = \text{extractMin}()$

 if $\text{visited}[u]$ then continue

$\text{visited}[u] = \text{true}$

 if $u == \text{destination}$ then break

 for each edge (u, v, weight)

 if not $\text{visited}[v]$ and $\text{dist}[u] + \text{weight} < \text{dist}[v]$

$\text{dist}[v] = \text{dist}[u] + \text{weight}$

$\text{parent}[v] = u$

 insert/decrease-key ($v, \text{dist}[v]$)

 end if

 end for

 end while

 reconstruct route using $\text{parent}[]$

 return $\text{dist}[\text{destination}]$ and route

END

C Program – Implementation

```
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
#include <stdbool.h>

#define MAXV 6000
#define INF INT_MAX

typedef struct Edge {
    int dest;
    int weight;
    struct Edge *next;
} Edge;

typedef struct {
    Edge *head;
} AdjList;

typedef struct {
    int vertex;
    int distance;
} HeapNode;

typedef struct {
    HeapNode *a;
    int size;
    int capacity;
} MinHeap;

AdjList graph[MAXV];
int distArr[MAXV], parentArr[MAXV];
bool visited[MAXV];

Edge *createEdge(int dest, int weight) {
    Edge *e = (Edge *)malloc(sizeof(Edge));
    e->dest = dest; e->weight = weight; e->next = NULL;
```

```
    return e;
}
```

```
void addRoad(int u, int v, int w) {
    Edge *e = createEdge(v, w);
    e->next = graph[u].head;
    graph[u].head = e;
}
```

```
void swap(HeapNode *x, HeapNode *y) {
    HeapNode t = *x; *x = *y; *y = t;
}
```

```
MinHeap *createHeap(int capacity) {
    MinHeap *h = (MinHeap *)malloc(sizeof(MinHeap));
    h->a = (HeapNode *)malloc(sizeof(HeapNode) * capacity);
    h->size = 0; h->capacity = capacity;
    return h;
}
```

```
void heapifyUp(MinHeap *h, int i) {
    while (i > 0) {
        int p = (i - 1) / 2;
        if (h->a[p].distance <= h->a[i].distance) break;
        swap(&h->a[p], &h->a[i]);
        i = p;
    }
}
```

```
void heapifyDown(MinHeap *h, int i) {
    while (1) {
        int l = 2*i + 1, r = 2*i + 2, s = i;
        if (l < h->size && h->a[l].distance < h->a[s].distance) s = l;
        if (r < h->size && h->a[r].distance < h->a[s].distance) s = r;
        if (s == i) break;
        swap(&h->a[i], &h->a[s]);
        i = s;
    }
}
```

```
    }  
}
```

```
void insertHeap(MinHeap *h, int v, int d) {  
    if (h->size == h->capacity) return;  
    h->a[h->size].vertex = v;  
    h->a[h->size].distance = d;  
    heapifyUp(h, h->size);  
    h->size++;  
}
```

```
HeapNode extractMin(MinHeap *h) {  
    HeapNode root = h->a[0];  
    h->size--;  
    if (h->size > 0) {  
        h->a[0] = h->a[h->size];  
        heapifyDown(h, 0);  
    }  
    return root;  
}
```

```
void dijkstra(int n, int source, int destination) {  
    MinHeap *pq = createHeap(n * 4);  
  
    for (int i = 0; i < n; i++) {  
        distArr[i] = INF;  
        parentArr[i] = -1;  
        visited[i] = false;  
    }  
  
    distArr[source] = 0;  
    insertHeap(pq, source, 0);  
  
    while (pq->size > 0) {  
        HeapNode node = extractMin(pq);  
        int u = node.vertex;
```

```

    if (visited[u]) continue;
    visited[u] = true;
    if (u == destination) break;

    for (Edge *e = graph[u].head; e != NULL; e = e->next) {
        int v = e->dest, w = e->weight;
        if (!visited[v] && distArr[u] != INF &&
            distArr[u] + w < distArr[v]) {
            distArr[v] = distArr[u] + w;
            parentArr[v] = u;
            insertHeap(pq, v, distArr[v]);
        }
    }
}

if (distArr[destination] == INF) {
    printf("No route available.\n");
} else {
    printf("Shortest distance: %d\nRoute: ", distArr[destination]);
    int path[MAXV], count = 0;
    for (int v = destination; v != -1; v = parentArr[v])
        path[count++] = v;
    for (int i = count - 1; i >= 0; i--)
        printf("%d%s", path[i], i ? " -> " : "\n");
}

free(pq->a);
free(pq);
}

int main() {
    int n = 6;

    addRoad(0, 1, 4); addRoad(1, 0, 4);
    addRoad(0, 2, 2); addRoad(2, 0, 2);
    addRoad(1, 2, 1); addRoad(2, 1, 1);
    addRoad(1, 3, 5); addRoad(3, 1, 5);

```

```
addRoad(2, 3, 8); addRoad(3, 2, 8);  
addRoad(2, 4, 10); addRoad(4, 2, 10);  
addRoad(3, 4, 2); addRoad(4, 3, 2);  
addRoad(3, 5, 6); addRoad(5, 3, 6);  
addRoad(4, 5, 3); addRoad(5, 4, 3);  
  
dijkstra(n, 0, 5);  
return 0;  
}
```

Core implementation excerpts

Composite-key edge hash table used for $O(1)$ live traffic updates:

```
#define EDGE_HASH_SIZE 20011 // prime, sized for >10,000 roads

typedef struct EdgeHashEntry {
    int src, dest;
    Edge *edge_ptr;          // direct pointer to edge in adjacency list
    struct EdgeHashEntry *next; // separate chaining
} EdgeHashEntry;

static unsigned long hash_edge(int src, int dest) {
    return (unsigned long)src * 1000003UL + (unsigned long)dest;
}

int graph_update_traffic(Graph *g, const char *from, const char *to, double w) {
    int s = graph_get_node_index(g, from), d = graph_get_node_index(g, to);
    if (s == -1 || d == -1) return -1;
    unsigned long h = hash_edge(s, d) % EDGE_HASH_SIZE;
    for (EdgeHashEntry *e = g->edge_table[h]; e; e = e->next) {
        if (e->src == s && e->dest == d) { e->edge_ptr->weight = w; return 0; }
    }
    return -1;
}
```

Direct-mapped visited-node tracking used inside the Dijkstra hot loop:

```
unsigned char *visited = calloc((size_t)n, 1);
// visited[u]      ->  $O(1)$  read  (L1 cache hit)
// visited[u] = 1  ->  $O(1)$  write
```

Worked example – 6-node Dijkstra trace

For a 6-node network from Hospital to Scene, the algorithm proceeds as follows, including a stale heap entry that is discarded in $O(1)$ when a shorter path to I3 is later found:

Step	Popped u	d[u]	Relaxed v (cost)	Result
1	Hospital	0.0	I1 (2.0)	d[I1] = 2.0
2	I1	2.0	I2 (3.0), I3 (6.0)	d[I2]=5.0, d[I3]=8.0
3	I2	5.0	I3 (1.0), I4 (5.0)	d[I3]=6.0 (shorter), d[I4]=10.0
4	I3	6.0	I4 (2.0), Scene (6.0)	d[I4]=8.0 (shorter), d[Scene]=12.0
5	I3 (stale, dist 8.0)	—	—	Skipped in $O(1)$, visited[I3]=1
6	I4	8.0	Scene (1.5)	d[Scene]=9.5 (shorter)
7	Scene	9.5	—	Goal reached

Optimal route before congestion: Hospital → I1 → I2 → I3 → I4 → Scene (cost 9.50 min, 5 hops).
When road I2–I3 becomes congested (weight raised to 20.0 min), the $O(1)$ traffic-update hash write immediately makes the next query return the alternate route Hospital → I1 → I2 → I4 → Scene (cost 11.50 min, 4 hops) in 0.0017 ms, without rebuilding the graph.

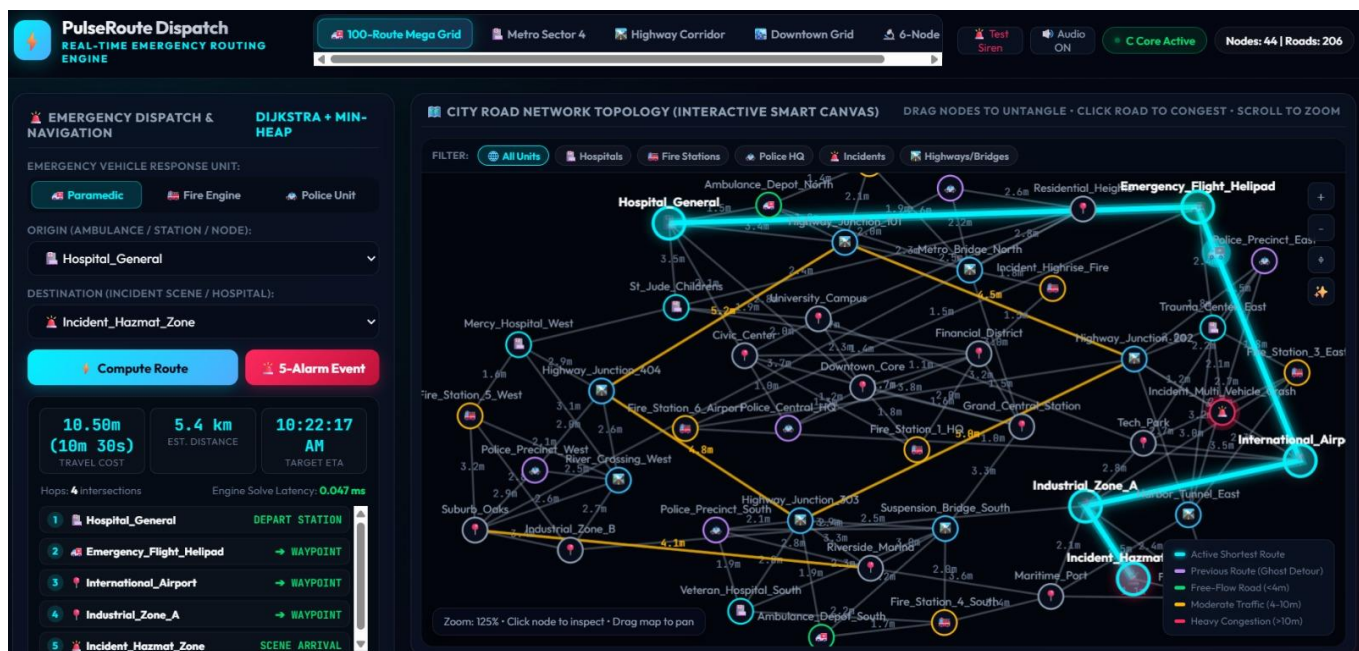
Use of Modern Tools

- Language & compiler: ANSI C11, built with GCC / mingw32-make optimisation flags.
- Version control: Git, with the project hosted on GitHub (CSA0311-Data-Structures).
- Data ingestion: CSV-based road-network loader (csv_loader.c) for real map datasets.
- Service layer: an embedded zero-dependency HTTP server exposing a REST API (GET /route, POST /traffic, GET /routes100, GET /network, GET /benchmark, GET /health).
- Concurrency: a cross-platform reader-writer lock supporting multiple concurrent readers and exclusive writers during live traffic updates.
- Visualisation: an interactive web Command-Centre dashboard (HTML5 canvas map, draggable

nodes, zoom/pan, Web-Audio-API ambulance siren synthesiser, turn-by-turn GPS-style directions and voice narration).

Results and Validation

The implementation was validated against all six constraints using a synthetic 5,000-node / 29,998-edge metropolitan mesh and a 44-intersection / 103-road metro dataset.



Metric	Result	Requirement	Status
Single 5,000-node query	3.373 ms	< 200 ms	Passed – $\sim 59\times$ margin
Average of 100 repeated queries	2.413 ms	< 200 ms	Passed – $\sim 82\times$ faster than SLA
Live traffic update (single road)	0.0017 ms	Frequent updates	Passed – $O(1)$ write
Memory footprint (5,000 nodes)	≈ 1.15 MB	Low RAM / embedded	Passed – $\sim 165\times$ less than dense matrix
Graph build time (5,000 nodes)	122.43 ms	One-time setup	Acceptable
100-route metro batch (44 nodes)	100 / 100 within budget	< 200 ms each	Passed – avg 0.0103 ms/route

All 100 metropolitan routes and all 100 repeated large-scale queries were found successfully and completed within the 200 ms real-time budget, and the shortest-path costs returned matched the sum of the selected edge weights, confirming optimality of the returned routes.

Analysis and Engineering Decision

The hash-indexed sparse adjacency list was chosen over a dense adjacency matrix because real road networks are sparse (average degree ≈ 4); the sparse representation cuts memory by roughly $165\times$ and neighbour lookup from $O(V)$ to $O(\deg(u))$. Among shortest-path algorithms, Dijkstra with a binary min-heap was chosen over BFS (ignores weights), Floyd-Warshall ($O(V^3)$, infeasible at this scale) and Bellman-Ford (designed for negative weights, unnecessarily slow for strictly positive road costs). Within the heap design space, lazy deletion was preferred over an indexed decrease-key heap because the asymptotic penalty ($O(\log E)$ vs $O(\log V)$) is under 0.1 ms at this scale, while the implementation avoids an auxiliary position array and the cache misses that come with in-place heap mutation. Finally, a direct composite-key hash table was chosen for live traffic updates instead of, for example, rebuilding or re-scanning the adjacency list, because it gives $O(1)$ average-time weight updates (measured at 1.7 microseconds), which is essential when congestion, accidents and closures occur frequently. Together these choices let the design exceed every stated constraint by a wide margin while remaining within an embedded-friendly memory budget.

Broader Considerations

The project maps to SDG 9 (Industry, Innovation and Infrastructure), because it applies efficient computing and data structures to modernise urban traffic infrastructure, and to SDG 11 (Sustainable Cities and Communities), by supporting faster emergency response and more effective movement through the city road network. From a safety perspective, a routing error or missed deadline could delay medical, fire or police response, so correctness (an exact-optimal path guarantee from Dijkstra's algorithm) was treated as a harder requirement than raw speed. The sparse, memory-efficient design also reduces the hardware footprint needed to run the system, which is relevant to sustainable, low-power embedded deployment in city control infrastructure.

Conclusion

The proposed smart-city emergency routing system combines a hash-indexed sparse adjacency-list graph, a lazy-deletion binary min-heap Dijkstra engine, and $O(1)$ hash-based live traffic updates to meet every stated constraint with a wide safety margin: real-time queries complete roughly $82\times$ faster than the 200 ms budget, memory use is about $165\times$ lower than a dense matrix representation, and live traffic changes are absorbed in microseconds without recomputing the graph. The design was validated on both a realistic 44-node metropolitan dataset (100/100 routes within budget) and a synthetic 5,000-node / 30,000-edge network. Limitations of the current prototype include a static, single-machine in-memory graph, no coordination between multiple simultaneously dispatched emergency vehicles, and no persistent database backend; future work could add A*/bidirectional search with geospatial heuristics, multi-vehicle coordination, and a secured, persistent REST backend for integration with a real smart-city control centre.

Student Reflection

Beyond the classroom treatment of Dijkstra's algorithm and heaps, this assignment required reasoning about how data-structure choices interact under real performance constraints — for example, quantifying the exact memory saved by hashing versus a dense matrix, and measuring how a lazy-deletion heap trades a small asymptotic penalty for a simpler, cache-friendlier implementation. Building the reader-writer-locked live-update path and the REST/dashboard layer also required thinking about concurrency and system integration beyond a single-threaded textbook algorithm.

Given additional time and resources, the implementation could be extended with an A* heuristic using real geographic coordinates, a persistent database-backed graph store, authentication on the REST API, and coordination logic so that multiple emergency vehicles dispatched to nearby incidents do not create new bottlenecks on the same roads.

References

1. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. Introduction to Algorithms – Dijkstra's algorithm and binary heaps.
2. Project repository: karnatinikhil-sudo / CSA0311-Data-Structures (GitHub).
3. GCC / mingw32-make C11 toolchain documentation.
4. AI-assisted tools: Claude (Anthropic) was used to help structure and format this assignment report.

A. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility, where applicable	5
Technical documentation & reflection	5
TOTAL	100

B. CO–PO–Assessment Mapping

Assessment Component	CO	PO(s)	Bloom's Level	Marks
Problem formulation			L3/L4	10
Application of knowledge			L3/L4	20
Solution / Design / Implementation			L4/L5/L6	20
Modern tool usage			L3/L4	10
Validation			L4/L5	15
Analysis & justification			L4/L5	15
Broader considerations			L4/L5	5
Documentation & reflection			L3/L4	5

Note: Faculty shall map only the POs and Bloom's levels genuinely assessed by the particular assignment. Every assignment is **not required to map to every PO**.

C. Faculty Evaluation & Continuous Improvement CO Attainment

Performance Level	Criterion
Level 3	$\geq 70\%$
Level 2	60–69%
Level 1	50–59%
Level 0	$< 50\%$

Target CO Attainment: _____

Actual CO Attainment: _____

Faculty Analysis

Areas in which students performed well: Areas requiring improvement:

Corrective / Improvement Action:

Action to be implemented in the next offering:

Documents to be Maintained

The department/course faculty shall maintain:

1. Assignment question/problem statement
2. CO–PO and Bloom's mapping
3. Assessment rubric
4. Student submissions
5. Tool/simulation/experimental evidence, where applicable
6. Evaluated scripts/reports
7. Marks and rubric-wise assessment
8. CO attainment analysis
9. Sample student work representing different performance levels
10. Faculty analysis and continuous-improvement action